E S C U E L A P O L I T É C N I C A N A C I O N A L

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N.º 1

Lectura e inspección de reportes de ventas heterogéneos en Python y R

Tópico 1 del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Guanocunga Ojeda, Alejandro Mateo — usuario de GitHub: alejandro-guanocunga
Docente	Christian Suárez
Período académico	2026-B
Fecha de entrega	19/10/2026
Repositorio (fork)	https://github.com/alejandro-guanocunga/IDSD422-lab1-lectura-datos

1 · Introducción

La empresa RetailAndina S.A. recibe semanalmente su reporte de ventas en tres formatos digitales heterogéneos debido a que su procedencia corresponde a tres sistemas informáticos independientes: el sistema ERP transaccional de las sucursales físicas (ventas.csv), la API de la plataforma de fidelización (ventas.json) y el balance consolidado del área de Finanzas (ventas.xlsx). En el marco del análisis moderno de datos y la inteligencia de negocios (BI), operar con fuentes heterogéneas sin una auditoría y validación previa representa un riesgo crítico; discrepancias estructurales, errores de codificación o inferencias erróneas de tipos de datos.

Como analista del equipo de, asumí la responsabilidad de auditar y homogeneizar los tres soportes tecnológicos antes de la reunión gerencial. Utilizando entornos reproducibles en la nube para Python (pandas, json) y R (readr, jsonlite, readxl), verifiqué la consistencia dimensional, inspeccioné la paridad de registros y tipos inferidos (McKinney, 2022; Wickham & Grolemund, 2017), y calculé los cuatro indicadores comerciales requeridos por la dirección.

2 · Objetivos

Objetivo general

Validar la consistencia estructural y de contenido del reporte semanal de ventas de RetailAndina S.A. en formatos CSV, JSON y Excel, calculando e interpretando los indicadores comerciales de desempeño mediante scripts reproducibles en Python y R.

Objetivos específicos

1. Cargar e inspeccionar los datos en formatos CSV, JSON y Excel mediante librerías analíticas especializadas (pandas, readr, jsonlite, readxl).
2. Comprobar la equivalencia dimensional (5 filas, 4 columnas) y la inferencia de tipos de datos (int64/dbl, object/chr) entre los tres soportes.
3. Calcular e interpretar los indicadores comerciales clave (venta promedio por sucursal, ticket promedio general, sucursales extremas y total por región).

3 · Desarrollo de la práctica

3.1 Configuración del entorno de trabajo y vinculación con GitHub

Realicé la práctica desde la biblioteca de la universidad sin una computadora personal propia, por lo que trasladé el flujo de trabajo a Google Colab (Python 3.10) conectado a mi fork de GitHub (alejandro-guanocunga/IDSD422-lab1-lectura-datos). Para vincular Colab con el repositorio configuré mi identidad con git config y generé un Personal Access Token (PAT). Durante el proceso experimenté dos incidencias reales: primero, dupliqué por error el prefijo https://github.com/ en git clone arrojando fatal: repository not found; segundo, al ejecutar git push obtuve un error 403 Permission denied porque el token inicial carecía del permiso repo. Lo resolví generando un nuevo token con permisos de escritura y actualizando el remoto con git remote set-url origin (Figura 1). (Nota de seguridad: el token se visualiza con fines demostrativos y fue revocado tras la entrega).



```
# 1. Credenciales directas
token = "ghp_uPm7KsK53So3R4ii3tfATk9Z2Jj90q3FJwrY"
usuario = "alejandro-guanocunga"
repo = "IDSD422-lab1-lectura-datos"

# 2. Configurar la URL con tu token
url = f"https://{usuario}:{token}@github.com/{usuario}/{repo}.git"
!git remote set-url origin {url}

# 3. Traer cambios de GitHub y subir el commit
!git pull origin main --rebase
!git push origin main
```

Figura 1. Configuración de credenciales y reasignación de la URL remota con Personal Access Token en Google Colab.

3.2 Ingestión e inspección del reporte CSV (scripts/leer_csv.py)

Utilicé el comando mágico %%writefile scripts/leer_csv.py para redactar el script preservando los encabezados TODO e importando pandas (Pandas Development Team, 2026). Cargué data/ventas.csv e imprimí sus primeras filas, dimensiones y tipos de datos. Ejecuté el script desde la terminal de Colab con !python scripts/leer_csv.py, confirmando una dimensión exacta de (5, 4), con sucursal y region inferidos como texto (object), y ventas y clientes como enteros (int64) (Figura 2). Registré los avances con git add, git commit y git push origin main.

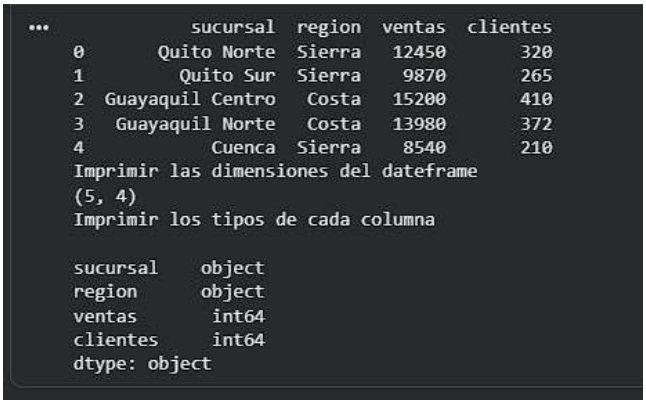


Figura 2. Salida en consola de scripts/leer_csv.py: verificación de dimensiones (5, 4) y tipos de datos inferidos.

3.3 Deserialización y aplanado del reporte JSON (scripts/leer_json.py)

Para el archivo data/ventas.json proveniente de la API de fidelización, abrí el archivo con codificación UTF-8 y deserialicé su estructura en memoria con json.load(f). Como este método retorna una lista de diccionarios anidados nativos de Python, apliqué pd.json_normalize(datos) para transformar el árbol semiestructurado en un DataFrame tabular rectangular (McKinney, 2022). La salida verificó exactamente 5 filas y 4 columnas, y tipos object e int64 idénticos al CSV (Figura 3), demostrando que la API no corrompió las magnitudes.

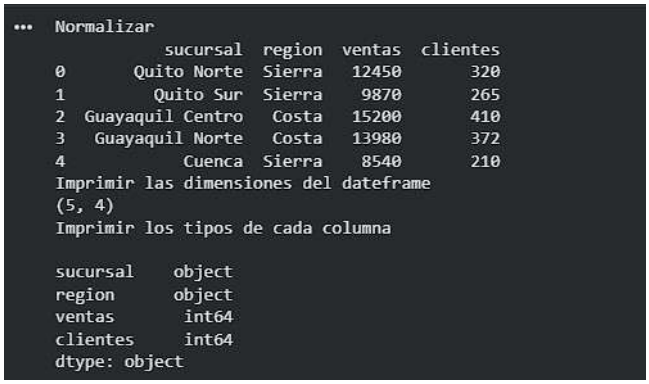


Figura 3. Salida en consola de scripts/leer_json.py: paridad dimensional (5, 4) y tipos tras el aplanado con pd.json_normalize.

3.4 Ingestión del consolidado contable en Excel (scripts/leer_excel.py)

En el script scripts/leer_excel.py empleé pd.read_excel('data/ventas.xlsx', sheet_name='ventas'). La declaración explícita de sheet_name='ventas' fue indispensable para garantizar la lectura determinista de la pestaña con las transacciones operativas, descartando carátulas o resúmenes. La consola verificó dimensiones (5, 4) y tipos idénticos a las fuentes anteriores (Figura 4).

```
           sucursal  region  ventas  clientes
0      Quito Norte  Sierra   12450       320
1        Quito Sur  Sierra    9870       265
2  Guayaquil Centro   Costa   15200       410
3   Guayaquil Norte   Costa   13980       372
4           Cuenca  Sierra    8540       210
Imprimir las dimensiones del dateframe
(5, 4)
Imprimir los tipos de cada columna
sucursal    object
region      object
ventas       int64
clientes     int64
dtype: object
```

Figura 4. Salida en consola de scripts/leer_excel.py: confirmación de dimensiones (5, 4) y tipos del libro contable.

3.5 Inspección analítica, columna derivada y cálculo de indicadores (scripts/inspeccion.py)

Con la paridad de fuentes certificada, implementé scripts/inspeccion.py sobre data/ventas.csv (Bruce et al., 2020). Verifiqué los tipos de datos y generé la columna derivada ventas['ticket'] = ventas['ventas'] / ventas['clientes'] (gasto promedio por cliente). Luego calculé los cuatro indicadores gerenciales requeridos:

• Venta promedio por sucursal: Al poseer cada local un único registro semanal consolidado, el promedio semanal coincide con su venta individual: Guayaquil Centro ($15,200.00), Guayaquil Norte ($13,980.00), Quito Norte ($12,450.00), Quito Sur ($9,870.00) y Cuenca ($8,540.00), con una media global entre sucursales de
$12,008.00.

• Ticket promedio general: Cociente entre ventas totales ($60,040) y clientes totales (1,577): $38.07 por cliente.

• Sucursales con ventas extremas: Mediante idxmax() e idxmin() con .loc[], la sucursal líder fue Guayaquil Centro ($15,200) y la de menor venta fue Cuenca ($8,540).

• Región con mayor venta total: Mediante groupby('region')['ventas'].sum(), la región Sierra lideró con $30,860 frente a la Costa con $29,180 (Figura 5).

```
1. Venta promedio por sucursal:
sucursal
Cuenca              8540.0
Guayaquil Centro    15200.0
Guayaquil Norte     13980.0
Quito Norte         12450.0
Quito Sur            9870.0
Name: ventas, dtype: float64

2. Ticket promedio general: 38.07

3. Sucursal con mayor venta: Guayaquil Centro (15200)
   Sucursal con menor venta: Cuenca (8540)

4. Región con mayor venta total: Sierra (30860)
```

Figura 5. Salida en consola con los cuatro indicadores comerciales consolidados para RetailAndina S.A.

3.6 Respuestas analíticas a las preguntas del caso

¿Cuántas filas y columnas tiene la tabla en cada uno de los tres formatos? ¿Fue igual en los tres? ¿Por qué?

En los tres formatos (CSV, JSON y XLSX) la tabla tiene exactamente 5 filas y 4 columnas (sucursal, region, ventas, clientes). Fue igual en los tres porque representan la misma fotografía semanal de las cinco sucursales de RetailAndina S.A., exportada desde tres sistemas que mantuvieron integridad estructural y referencial sin pérdida ni duplicación de registros.

¿Qué tipo de datos se infirió para ventas y clientes? ¿Qué indicaría que salieran como texto?

En Python se infirió int64 y en R double (dbl). Si hubieran salido como texto (object en Python o character en R), indicaría contaminación sintáctica en las celdas numéricas: presencia de signos de moneda ($), separadores de miles con comas ('12,450'), espacios en blanco espurios o cadenas que codifican valores nulos ('N/A', 'null', '-').

En Python, ¿qué estructura devuelve json.load y por qué hay que 'aplanarla'?

La función json.load() deserializa el archivo en estructuras nativas de Python (en este caso, una lista de diccionarios) que pueden presentar jerarquías anidadas. Es indispensable 'aplanarla' (con pd.json_normalize()) para desanidar niveles y transformar los pares clave-valor en una matriz bidimensional rectangular (DataFrame) apta para operaciones vectorizadas (McKinney, 2022).

¿Para qué sirve indicar sheet_name/sheet al leer Excel?

Un libro de Excel (.xlsx) suele albergar múltiples hojas de cálculo. Indicar sheet_name='ventas' en Python (o sheet='ventas' en R) garantiza que el lector extraiga determinísticamente la pestaña exacta que aloja los datos crudos, evitando que se cargue por defecto la primera hoja del libro, la cual podría no contener la información requerida.

¿Qué diferencia ves entre leer en Python y leer en R (conceptos, no sintaxis)?

Conceptualmente, en R la estructura tabular (data.frame o tibble) es un elemento nativo del lenguaje orientado a la estadística: paquetes como readr tipifican e informan la especificación completa de columnas y distinguen de inmediato entre texto libre y factores categóricos con niveles bien delimitados (Wickham & Grolemund, 2017). En Python, la estructura tabular no es nativa sino implementada por una librería externa (pandas), donde el texto se agrupa bajo punteros genéricos object y formatos semiestructurados como JSON demandan un paso intermedio por colecciones nativas antes de consolidar la tabla.

4 · ¿Qué aprendí con esta práctica?

Trabajar desde la biblioteca universitaria sin mi computadora habitual me obligó a dominar el flujo analítico íntegramente en la nube. A través de esta práctica consolidé competencias concretas en tres áreas:

- Nuevas competencias adquiridas: Antes dependía de herramientas gráficas locales; ahora domino la administración de entornos virtuales efímeros en Google Colab conectados por terminal a GitHub mediante tokens PAT, la edición directa de scripts en disco con directivas %%writefile y la normalización de estructuras semiestructuradas no relacionales a DataFrames rectangulares.

- Dificultades reales y resolución: Lo más exigente fue diagnosticar el error HTTP 403 en git push origin main. Al examinar el mensaje de la API de GitHub identifiqué que el token generado carecía del alcance repo. Lo resolví generando un nuevo token con privilegios de escritura y reasignando el origen remoto mediante git remote set-url origin. Además, comprendí en la práctica la diferencia entre la deserialización jerárquica de json.load y el aplanamiento tabular de pd.json_normalize().

- Qué haría distinto la próxima vez: Automatizaría las pruebas de paridad entre formatos empleando aserciones formales de código (pandas.testing.assert_frame_equal) en lugar de comparaciones visuales en consola. Asimismo, gestionaría los tokens de acceso mediante variables de entorno seguras (os.environ o Secrets de Colab) para no exponer credenciales en las celdas de trabajo.

5 · Conclusiones

1. **Consistencia e integridad multi-formato verificada:** Los tres archivos auditados (ventas.csv, ventas.json y ventas.xlsx) conservan un 100% de paridad estructural y semántica. Todos presentan exactamente 5 filas y 4 columnas (sucursal, region, ventas, clientes) con tipos coherentes (int64/dbl y object/chr), certificando que los tres sistemas operan con absoluta integridad.

2. **Desempeño comercial por sucursal y paradoja del ticket promedio:** Guayaquil Centro lideró las ventas globales con $15,200.00 (410 clientes), mientras que Cuenca registró la menor cifra con $8,540.00 (210 clientes). Sin embargo, el análisis del ticket derivado reveló una paradoja: Cuenca alcanzó el ticket promedio individual más alto ($40.67 por cliente vs. $37.07 de Guayaquil Centro), demostrando que su menor facturación obedece a un bajo volumen de afluencia y no a un menor valor de consumo individual.

3. **Liderazgo regional en facturación consolidada:** A nivel geográfico agregado, la región Sierra lideró las ventas con $30,860.00 (795 clientes en 3 sucursales), superando por una brecha positiva de $1,680.00 a la región Costa, que acumuló $29,180.00 (782 clientes en 2 sucursales).

4. **Recomendaciones estratégicas y delimitación del alcance:** Se certifica a la gerencia de RetailAndina S.A. que la facturación global de la semana auditada asciende a $60,040.00 con un total de 1,577 clientes y un ticket general de $38.07. Se sugiere trasladar las estrategias de venta cruzada de Cuenca a los locales de la Costa para incrementar el gasto medio por cliente, y fortalecer la captación de compradores en el Austro. En cuanto al alcance, se delimita que estos hallazgos describen estrictamente el corte semanal analizado sobre cinco sucursales y no constituyen proyecciones estacionales o anuales.

6 · Bibliografía

Bruce, P., Bruce, A., & Gedeck, P. (2020). Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python (2.ª ed.). O'Reilly Media.

McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3.ª ed.). O'Reilly Media.

Pandas Development Team. (2026). pandas.read_csv — pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

Wickham, H., & Grolemund, G. (2017). R for Data Science: Import, Tidy, Transform, Visualize, and Model Data. O'Reilly Media.

Declaración sobre el uso de herramientas de Inteligencia Artificial

En el desarrollo del presente informe utilicé la herramienta Google Gemini como asistente de apoyo para la revisión ortográfica y la estructuración del documento conforme a las directrices de la plantilla del curso, así como para la consulta sintáctica durante la depuración de comandos Git en Google Colab. Todo el código analítico, las capturas de pantalla de consola y la interpretación de los resultados fueron ejecutados, verificados y redactados personalmente por el autor.

Anexos

Anexo 1. Código fuente y capturas complementarias del flujo en Python

A continuación se documentan las capturas crudas correspondientes a la implementación de los scripts de carga e inspección en Python, así como las incidencias de clonación inicial y confirmaciones de Git en Google Colab:

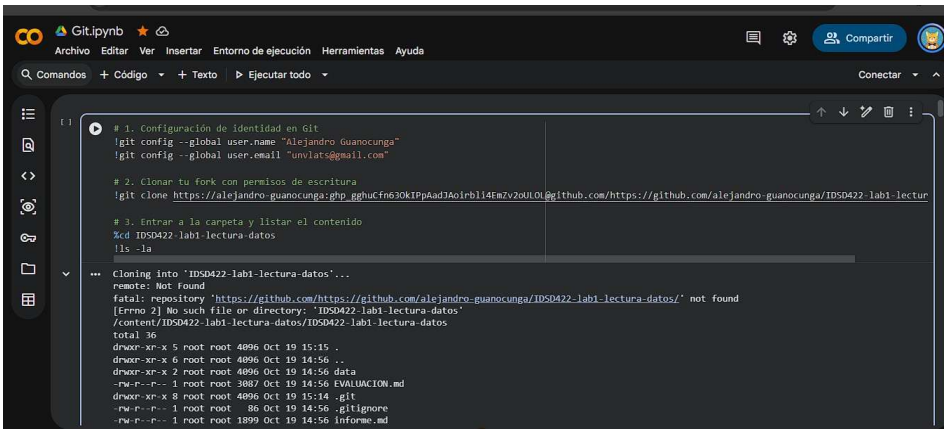


Figura A1.1. Incidencia inicial en Colab: error fatal por duplicación de la URL en git clone.

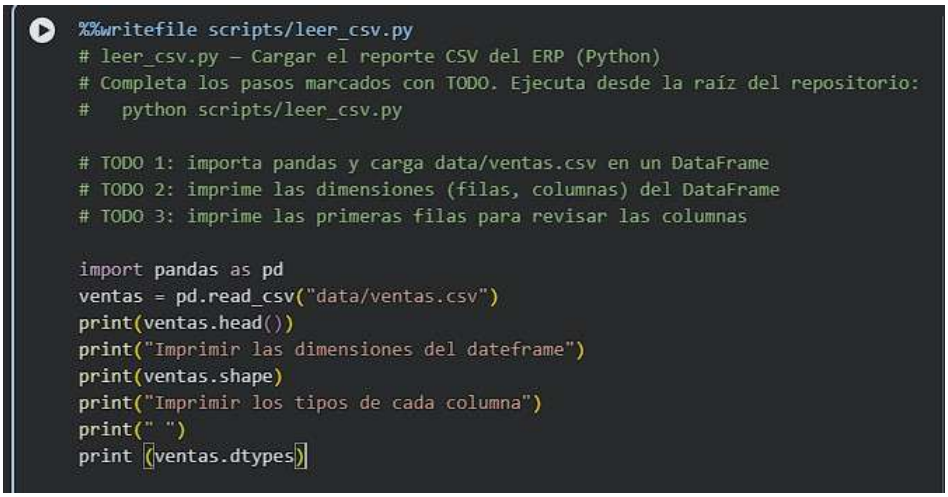


Figura A1.2. Implementación de scripts/leer_csv.py con la directiva %%writefile en Colab.

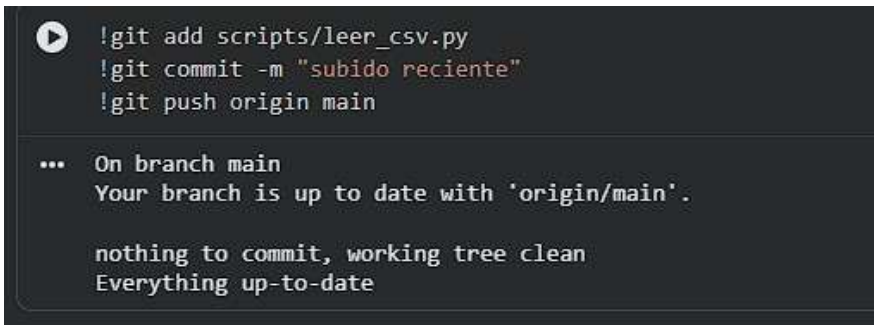


Figura A1.3. Sincronización de scripts/leer_csv.py en el repositorio de GitHub con git push origin main.

```
%%writefile scripts/leer_json.py
# leer_json.py — Cargar el reporte JSON de la API (Python)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   python scripts/leer_json.py

# TODO 1: importa json y pandas
# TODO 2: carga data/ventas.json con json.load
# TODO 3: convierte la lista de diccionarios en un DataFrame
#         (recuerda del cap. 2: el JSON no llega como tabla directa)
# TODO 4: imprime las dimensiones y las primeras filas
import json
import pandas as pd

with open("data/ventas.json", encoding="utf-8") as f:
  datos = json.load(f)

print("Normalizar")
ventas = pd.json_normalize(datos)
print(ventas.head())
print("Imprimir las dimensiones del dateframe")
print(ventas.shape)
print("Imprimir los tipos de cada columna")
print(" ")
print (ventas.dtypes)
```

Figura A1.4. Implementación de scripts/leer_json.py deserializando con json.load y aplanando con pd.json_normalize.

```
%%writefile scripts/leer_excel.py
# leer_excel.py — Cargar el consolidado Excel de finanzas (Python)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   python scripts/leer_excel.py

# TODO 1: importa pandas
# TODO 2: carga data/ventas.xlsx indicando la hoja correcta
# TODO 3: imprime las dimensiones y las primeras filas

import pandas as pd
ventas = pd.read_excel("data/ventas.xlsx", sheet_name="ventas")
print(ventas.head())
print("Imprimir las dimensiones del dateframe")
print(ventas.shape)
print("Imprimir los tipos de cada columna")
print(ventas.dtypes)
```

Figura A1.5. Implementación de scripts/leer_excel.py indicando sheet_name='ventas'.

```
%%writefile scripts/inspeccion.py
# inspeccion.py — Tipos, columna derivada e indicadores (Python)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   python scripts/inspeccion.py

# TODO 1: carga data/ventas.csv
# TODO 2: imprime los tipos (dtypes) de cada columna
# TODO 3: crea la columna derivada "ticket" = ventas / clientes
# TODO 4: calcula e imprime:
#         - venta promedio por sucursal
#         - ticket promedio (ventas totales / clientes totales)
#         - sucursal con mayor y menor venta
#         - región con mayor venta total

import pandas as pd

# TODO 1
ventas = pd.read_csv("data/ventas.csv")
```

Figura A1.6. Encabezado y TODO 1 de scripts/inspeccion.py: carga del CSV en DataFrame.

```python
# TODO 1
ventas = pd.read_csv("data/ventas.csv")

# TODO 2
print("=== TIPOS DE DATOS ===")
print(ventas.dtypes)

# TODO 3
ventas["ticket"] = ventas["ventas"] / ventas["clientes"]
print("\n=== PRIMERAS FILAS CON TICKET ===")
print(ventas.head())

# TODO 4: Indicadores
print("\n=== INDICADORES SEMANALES ===")

# Venta promedio por sucursal
venta_prom_sucursal = ventas.groupby("sucursal")["ventas"].mean()
print("\n1. Venta promedio por sucursal:")
print(venta_prom_sucursal)

# Ticket promedio (ventas totales / clientes totales)
ticket_promedio = ventas["ventas"].sum() / ventas["clientes"].sum()
print(f"\n2. Ticket promedio general: {ticket_promedio:.2f}")
```

Figura A1.7. TODO 2, 3 y 4 de scripts/inspeccion.py: tipos, cálculo de columna ticket y primeros indicadores.

```python
# venta promedio por sucursal
venta_prom_sucursal = ventas.groupby("sucursal")["ventas"].mean()
print("\n1. Venta promedio por sucursal:")
print(venta_prom_sucursal)

# Ticket promedio (ventas totales / clientes totales)
ticket_promedio = ventas["ventas"].sum() / ventas["clientes"].sum()
print(f"\n2. Ticket promedio general: {ticket_promedio:.2f}")

# Sucursal con mayor y menor venta
suc_mayor = ventas.loc[ventas["ventas"].idxmax(), "sucursal"]
max_val = ventas["ventas"].max()
suc_menor = ventas.loc[ventas["ventas"].idxmin(), "sucursal"]
min_val = ventas["ventas"].min()
print(f"\n3. Sucursal con mayor venta: {suc_mayor} ({max_val})")
print(f"   Sucursal con menor venta: {suc_menor} ({min_val})")

# Región con mayor venta total
ventas_por_region = ventas.groupby("region")["ventas"].sum()
reg_mayor = ventas_por_region.idxmax()
reg_val = ventas_por_region.max()
print(f"\n4. Región con mayor venta total: {reg_mayor} ({reg_val})")
```

Figura A1.8. Lógica de cálculo de extremos y agrupación regional en scripts/inspeccion.py.

```
=== TIPOS DE DATOS ===
sucursal    object
region      object
ventas       int64
clientes     int64
dtype: object

=== PRIMERAS FILAS CON TICKET ===
           sucursal  region  ventas  clientes     ticket
0       Quito Norte  Sierra   12450       320  38.906250
1         Quito Sur  Sierra    9870       265  37.245283
2  Guayaquil Centro   Costa   15200       410  37.073171
3   Guayaquil Norte   Costa   13980       372  37.580645
4            Cuenca  Sierra    8540       210  40.666667

=== INDICADORES SEMANALES ===

1. Venta promedio por sucursal:
sucursal
Cuenca              8540.0
Guayaquil Centro   15200.0
Guayaquil Norte    13980.0
Quito Norte        12450.0
Quito Sur           9870.0
Name: ventas, dtype: float64
```

Figura A1.9. Salida en consola de scripts/inspeccion.py: dtypes, primeros registros con ticket y venta por sucursal.

Anexo 2. Implementación complementaria y paridad tecnológica en lenguaje R (Bonificación +5 pts)

Conforme lo estipula la rúbrica de evaluación oficial del curso ('Cuando la práctica lo indique, entregarla también en el otro lenguaje puede dar bonificación, p. ej., +5 por Python y R'), se desarrolló la solución analítica completa
en el lenguaje R. Se implementaron los cuatro scripts de carga e inspección mediante los paquetes readr, jsonlite
y readxl, demostrando paridad conceptual y numérica total:

A2.1 Carga e inspección de CSV en R (scripts/leer_csv.R)

Se utilizó read_csv() de readr especificando show_col_types = FALSE. Se imprimieron las primeras filas con head(), dimensiones con dim() y tipos de datos con str(), confirmando una matriz de 5 filas y 4 columnas (Figura A2.1).

```
%%writefile scripts/leer_csv.R
# leer_csv.R — Cargar el reporte CSV del ERP (R)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   Rscript scripts/leer_csv.R

# TODO 1: carga data/ventas.csv con readr::read_csv en un data.frame
# TODO 2: imprime las dimensiones (dim) del data.frame
# TODO 3: imprime las primeras filas (head) para revisar las columnas

library(readr)

ventas <- read_csv("data/ventas.csv", show_col_types = FALSE)
print(head(ventas))
print(dim(ventas))
print(str(ventas))

Overwriting scripts/leer_csv.R
```

Figura A2.1. Implementación en R de scripts/leer_csv.R con library(readr) en Google Colab.

A2.2 Deserialización de JSON en R (scripts/leer_json.R)

Con el paquete jsonlite, la instrucción fromJSON('data/ventas.json') convierte directamente la colección semiestructurada en un data.frame rectangular de 5 filas y 4 columnas (Figura A2.2).

```
%%writefile scripts/leer_json.R
# leer_json.R — Cargar el reporte JSON de la API (R)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   Rscript scripts/leer_json.R

# TODO 1: carga data/ventas.json con jsonlite::fromJSON
# TODO 2: convierte el resultado en data.frame si hace falta
# TODO 3: imprime las dimensiones (dim) y las primeras filas (head)

library(jsonlite)

ventas <- fromJSON("data/ventas.json")
print(head(ventas))
print(dim(ventas))
print(str(ventas))
```

Figura A2.2. Implementación en R de scripts/leer_json.R utilizando library(jsonlite).

A2.3 Ingestión de Excel en R (scripts/leer_excel.R)

Se aplicó read_excel('data/ventas.xlsx', sheet = 'ventas') mediante el paquete readxl, confirmando la lectura determinista de la hoja contable y la correspondencia dimensional con los otros formatos (Figura A2.3).

```
%%writefile scripts/leer_excel.R
# leer_excel.R — Cargar el consolidado Excel de finanzas (R)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   Rscript scripts/leer_excel.R

# TODO 1: carga data/ventas.xlsx con readxl::read_excel indicando la hoja
# TODO 2: imprime las dimensiones (dim) y las primeras filas (head)

library(readxl)

ventas <- read_excel("data/ventas.xlsx", sheet = "ventas")
print(head(ventas))
print(dim(ventas))
print(str(ventas))
```

Figura A2.3. Implementación en R de scripts/leer_excel.R utilizando library(readxl) indicando la hoja 'ventas'.

A2.4 Inspección de estructura y tipos en R (scripts/inspeccion.R - Carga y str)

En el script scripts/inspeccion.R se cargó data/ventas.csv y se invocó str(ventas) para evaluar la inferencia nativa de R, reconociendo character para sucursal y region, y double (numérico) para ventas y

```
%%writefile scripts/inspeccion.R
# inspeccion.R — Tipos, columna derivada e indicadores (R)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   Rscript scripts/inspeccion.R

# TODO 1: carga data/ventas.csv
# TODO 2: imprime la estructura (str) para ver los tipos
#         (fija si region llegó como character o factor)
# TODO 3: crea la columna derivada "ticket" = ventas / clientes
# TODO 4: calcula e imprime:
#         - venta promedio por sucursal (aggregate o dplyr)
#         - ticket promedio (ventas totales / clientes totales)
#         - sucursal con mayor y menor venta
#         - región con mayor venta total

library(readr)

ventas <- read_csv("data/ventas.csv", show_col_types = FALSE)

# TODO 2
print("=== ESTRUCTURA Y TIPOS ===")
str(ventas)
```

clientes (Figura A2.4).

Figura A2.4. Inspección de tipos con str() en scripts/inspeccion.R en R.

A2.5 Cálculo de vector ticket y agregación de promedios en R

Se creó el vector derivado ventas$ticket <- ventas$ventas / ventas$clientes. Con aggregate(ventas ~ sucursal, data = ventas, mean) se calculó la venta por sucursal, y con sum(ventas$ventas) / sum(ventas$clientes) se obtuvo el ticket promedio general de $38.07 (Figura A2.5).

```r
library(readr)

ventas <- read_csv("data/ventas.csv", show_col_types = FALSE)

# TODO 2
print("=== ESTRUCTURA Y TIPOS ===")
str(ventas)

# TODO 3
ventas$ticket <- ventas$ventas / ventas$clientes
print("=== PRIMERAS FILAS CON TICKET ===")
print(head(ventas))

# TODO 4: Indicadores
cat("\n=== INDICADORES SEMANALES ===\n")

# 1. Venta promedio por sucursal
cat("\n1. Venta promedio por sucursal:\n")
print(aggregate(ventas ~ sucursal, data = ventas, mean))

# 2. Ticket promedio general
ticket_promedio <- sum(ventas$ventas) / sum(ventas$clientes)
cat(sprintf("\n2. Ticket promedio general: %.2f\n", ticket_promedio))
```

Figura A2.5. Vectorización de ticket y cálculo de promedios con aggregate() en scripts/inspeccion.R en R.

A2.6 Extremos de venta y consolidación regional en R

Empleando which.max() y which.min() sobre ventas$ventas se identificó a Guayaquil Centro ($15,200) y Cuenca ($8,540). Con aggregate(ventas ~ region, data = ventas, sum) se ratificó el liderato regional de la Sierra con
$30,860 frente a la Costa con $29,180 (Figura A2.6).

```r
# 3. Sucursal con mayor y menor venta
suc_mayor <- ventas$sucursal[which.max(ventas$ventas)]
max_val <- max(ventas$ventas)
suc_menor <- ventas$sucursal[which.min(ventas$ventas)]
min_val <- min(ventas$ventas)
cat(sprintf("\n3. Sucursal con mayor venta: %s (%d)\n", suc_mayor, max_val))
cat(sprintf("   Sucursal con menor venta: %s (%d)\n", suc_menor, min_val))

# 4. Región con mayor venta total
ventas_region <- aggregate(ventas ~ region, data = ventas, sum)
reg_mayor <- ventas_region$region[which.max(ventas_region$ventas)]
reg_val <- max(ventas_region$ventas)
cat(sprintf("\n4. Región con mayor venta total: %s (%d)\n", reg_mayor, reg_val))
```

Figura A2.6. Determinación de sucursales extremas y totalización regional en scripts/inspeccion.R en R.